

Anthropic Guidance Principles

10 core principles Anthropic uses to construct every correct exam answer. Memorise the logic, not just the answer.

Defence in depth · Minimal footprint · Simplest fix first · Programmatic enforcement · Structured errors · Graceful degradation · Human oversight · Policy gap escalation · Independent review · Tool descriptions

The 10 Anthropic Guidance Principles

The logic behind every correct exam answer

1 Defence in Depth — Validate at the Most Fundamental Layer

Critical compliance rules must be enforced inside the tool's own backend code — not only in the layer that calls it. Orchestration hooks, system prompts, and prerequisite gates are additional layers, not the primary one. If a tool is called directly (bypassing orchestration), backend validation still runs. Hooks do not.

EXAM SIGNAL: '100% compliance', 'zero margin for failure', 'deterministic guarantee', 'regulatory requirement' → **backend validation in tool code wins over hooks and prompts.**

2 Minimal Footprint — Least Privilege, Reversible Actions

Agents should request only the permissions they need for the current task. Prefer reversible over irreversible actions. Do less when uncertain. Never autonomously escalate permissions — surface ambiguity to the user and request elevation explicitly.

EXAM SIGNAL: 'Agent needs more access', 'irreversible action', 'autonomous expansion' → **request permission / confirm with user, never self-escalate.**

3 Simplest Fix First — Don't Over-Engineer

Fix the problem at the lowest applicable layer with the simplest mechanism that works. Don't add architectural complexity when a prompt or description change solves it. Adding a multi-agent classifier when rewriting a tool description fixes tool misrouting is over-engineering.

EXAM SIGNAL: Tool misrouting → **fix description (not routing classifier).** Format issues → **add examples (not new validation).** The exam rewards precision over sophistication.

4 Programmatic Enforcement for Critical Paths

Financial operations, security checks, regulatory compliance, and identity verification must be enforced by code — not by prompt instructions. Prompt guidance is probabilistic (~90–95%). Code-level gates are deterministic (100%). For critical paths, anything less than 100% is unacceptable. 'Strengthen the system prompt' is always wrong here.

EXAM SIGNAL: 'Financial', 'security', 'compliance', 'regulatory', 'HIPAA', 'PCI-DSS' → **programmatic gate or backend validation. Never strengthen the prompt.**

5 Structured Errors with Full Metadata

Every tool error must include: errorCategory (transient/validation/business/permission) + isRetryable boolean + human-readable description. Generic errors leave agents unable to make recovery decisions. Auth errors and business rule violations are never retryable. Network timeouts are retryable.

EXAM SIGNAL: 'Agent retries incorrectly', 'agent doesn't know what went wrong' → **add structured error metadata.** 'Always retry on failure' is always wrong.

6

Graceful Degradation — Never Terminate All on Partial Failure

When one component fails in a multi-agent pipeline, continue with what succeeded. Return partial results with gap annotation. Never silently fail or hard-terminate the entire workflow. The coordinator must receive: failure type + what was attempted + partial results + alternative approaches.

EXAM SIGNAL: 'One subagent fails' → continue others, annotate gap, return partial results. Never: 'terminate all' or 'ignore the failure'.

7

Human Oversight Before Irreversible High-Stakes Actions

In regulated environments (healthcare, finance, legal), AI confidence scores never replace human sign-off for irreversible actions. Even at 99% confidence — pause and confirm before committing. Autonomous action on high-stakes decisions is always wrong in regulated contexts.

EXAM SIGNAL: 'Healthcare', 'regulated', '99% confident', 'irreversible action' → require human checkpoint before proceeding, regardless of AI confidence.

8

Escalate on Policy Gaps — Never Improvise Outside Instructions

When a situation isn't covered by the agent's instructions, escalate immediately. Do not interpolate policy, infer intent, or 'use judgment.' Novel situations require human decision-making. Complex cases that ARE covered by policy should be handled autonomously, however difficult they are.

EXAM SIGNAL: 'No instructions for this case' → escalate. 'Complex but within policy' → handle it. Complexity alone is never an escalation trigger.

9

Independent Review for Logical Errors — Not Same-Session Self-Critique

The same reasoning context that produced an error will rationalise it during review. For catching logical errors, reasoning mistakes, and factual inaccuracies — use a separate Claude instance with fresh context. Self-critique within the same session is for pre-delivery completeness checks, not error detection.

EXAM SIGNAL: 'Logical errors', 'wrong conclusions', 'factual mistakes' → separate instance. 'Incomplete responses, varies per case' → self-critique step.

10

Tool Descriptions Are the Primary Routing Mechanism

Claude selects tools based on their descriptions. Wrong tool called = description problem. Never add routing classifiers, system prompt instructions about tool selection, or tool_choice hacks before fixing the description. The description is the root layer for tool selection — fix it first, always.

EXAM SIGNAL: 'Claude calls the wrong tool' → rewrite description. Never: add classifier, change tool_choice, add prompt instructions about tool selection.

Enforcement Hierarchy — Hook vs Backend vs Prompt

The nuance the exam tests most

This is the subtlest enforcement question. PreToolCall hooks ARE deterministic — they DO block calls. But they run in the orchestration layer, not inside the tool. If orchestration is bypassed, the hook is skipped. Backend validation cannot be bypassed.

Mechanism	Runs in	Bypassed if...	Use for	Anthropic recommendation
Backend validation in tool code	Inside tool's execution function	Cannot be bypassed — always runs	Critical compliance rules, zero-tolerance checks, regulatory requirements	PRIMARY layer for critical rules
Programmatic prerequisite gate	Code that blocks Tool B until Tool A completes	Only if gate code is removed	Workflow order enforcement (verify identity → process refund)	Required for financial / security

PreToolCall hook	Orchestration layer — BEFORE tool executes	Tool called directly / orchestration misconfigured	Rate limiting, logging, secondary validation, audit trail	Additional layer — NOT the primary one
PostToolCall hook	Orchestration layer — AFTER tool executes	Same as PreToolCall	Output validation, data stripping, logging completion	Additional layer — NOT the primary one
System prompt instruction	Prompt layer — model reads and follows	Model occasionally doesn't follow (~5–10% failure rate)	Behavioural guidance, low-stakes preferences, style	NEVER for compliance or critical rules

KEY Hierarchy (most → least robust): Backend validation > Programmatic gate > PreToolCall hook > System prompt. 'Zero tolerance / 100% compliance / deterministic' → backend validation always wins.

Enforcement Question Cheat Card — Which Answer Wins When

Question asks for...	Correct answer	Eliminate these distractors
100% compliance / zero tolerance / deterministic / regulatory	Backend validation inside tool's execution code	Hooks (bypassed if orchestration skipped), system prompt (probabilistic)
Workflow order enforcement (A must precede B)	Programmatic prerequisite gate in code	System prompt 'always do A before B' (probabilistic, ~5% failure)
Audit trail / rate limiting / secondary check on every tool call	PreToolCall / PostToolCall hook	Backend validation (overkill for logging), system prompt (unreliable)
Low-stakes behavioural preference / style / format	System prompt instructions	Programmatic gates (overkill), hooks (unnecessary complexity)
Catch logical errors in output	Independent review — separate Claude instance	Same-session self-critique, hooks, backend validation
Completeness varies per case — sometimes missing context	Self-critique step with completeness checklist	Few-shot (fixed template), independent review (for errors, not completeness)
Escalation decision in support agent	3 valid triggers: explicit request / policy gap / can't progress	Sentiment (frustrated customer) / self-confidence / complexity alone
High-stakes action in regulated environment	Human checkpoint before irreversible action	'AI is 99% confident — proceed autonomously'

Escalation Triggers — 3 Valid, 3 Invalid (Memorise Both Lists)

■ INVALID triggers — eliminate if answer uses these

- Customer seems frustrated or upset (sentiment)
- Claude's confidence score is low (self-assessment)
- The case is complex (complexity alone ≠ escalate)
- Claude is 'not sure' about the answer
- The case is unusual or unfamiliar

■ VALID triggers — only these 3

- Customer explicitly requests a human — always honour, no exceptions
- Policy gap exists — no instructions cover this situation
- Unable to make meaningful progress after genuine attempts

TRAP — COMPLEXITY TRAP	'The request is complex, so escalate.' Wrong. Complex but in-policy = handle it. Only escalate when policy is SILENT — not when the case is merely difficult.
TRAP — CONFIDENCE TRAP	'Claude isn't confident, so escalate.' Wrong. Confidence is not calibrated to accuracy. Claude can be confidently wrong and uncertain about correct answers. Use programmatic criteria.

Self-Critique vs Independent Review — 3-Type Decision Table

Review type	When to use	Problem it solves	Exam signal words
Self-critique step (same session, pre-delivery)	Output is correct but inconsistently complete — sometimes missing policy context, timelines, next steps	Case-specific completeness gaps that vary per response — adapts to each case	'sometimes omits', 'inconsistently explains', 'varies by case', 'correct but incomplete'
Independent review (separate instance, fresh context)	Need to catch logical errors, factual mistakes, reasoning errors the generator rationalised	Errors the same reasoning context would approve — catches what generator rationalised away	'logical errors', 'wrong conclusions', 'factual mistakes', 'reasoning errors'
Few-shot examples (2–4)	Output format, tone, or length is inconsistent across runs — not a completeness problem	Format/structure variance across multiple calls — provides behavioural anchor	'varies in format', 'inconsistent tone', 'different length each time', 'wrong structure'

KEY The key distinction: self-critique adapts to what's missing in THIS specific response (completeness). Independent review catches errors the same reasoning would approve (logical correctness). Few-shot anchors format across ALL responses (consistency).

Anthropic Guidance Principles complete. For layer-by-layer fix reference, see CCA-F_Layers_Fix_At_Right_Layer.pdf